OPERATING SYSTEMS [BCE26PC01]

UNIT-4

PRAVIN S. GAME, PH.D.
ASSOCIATE PROFESSOR
DEPARTMENT OF COMPUTER ENGINEERING
PIMPRI CHINCHWAD COLLEGE OF ENGINEERING

- **Course Objectives:**

1. To introduce the Operating system and process, threads and CPU scheduling

2. To explore inter process communication mechanisms, to introduce the critical-section problem and solutions.

3. To explore various techniques of allocating memory to processes and explain the concepts of demand paging, page-replacement algorithms.

4. **To describe the details of implementing local file systems and directory structures**

- **Course Outcomes:**

After learning the course, the students should be able to:

1. Compare various process scheduling algorithms for a given snapshot of the system.
2. Analyze IPC mechanisms and solutions of process synchronization for critical section problems.
3. Analyze the performance of memory management algorithms for a given problem.
4. **Apply disk scheduling policies for a given I/O request sequence with file management concepts**

UNIT - 4

- **I/O Management:**

Concept of Files, File Allocation Methods, Free-Space Management.

Disk Scheduling- FIFO, SSTF, SCAN, C-SCAN.

FA 2

- Problem solving
- Unit 3 and 4 : One question on each unit

Sr. No	Performance Indicator	Marks	Excellent (3)	Fair (1-2)
1	Solution	3	Exact Solution with all logical conclusions	Partial correct solution
2	Step by step presentation	2	All steps and diagrams are correctly written.	Partial steps and diagrams

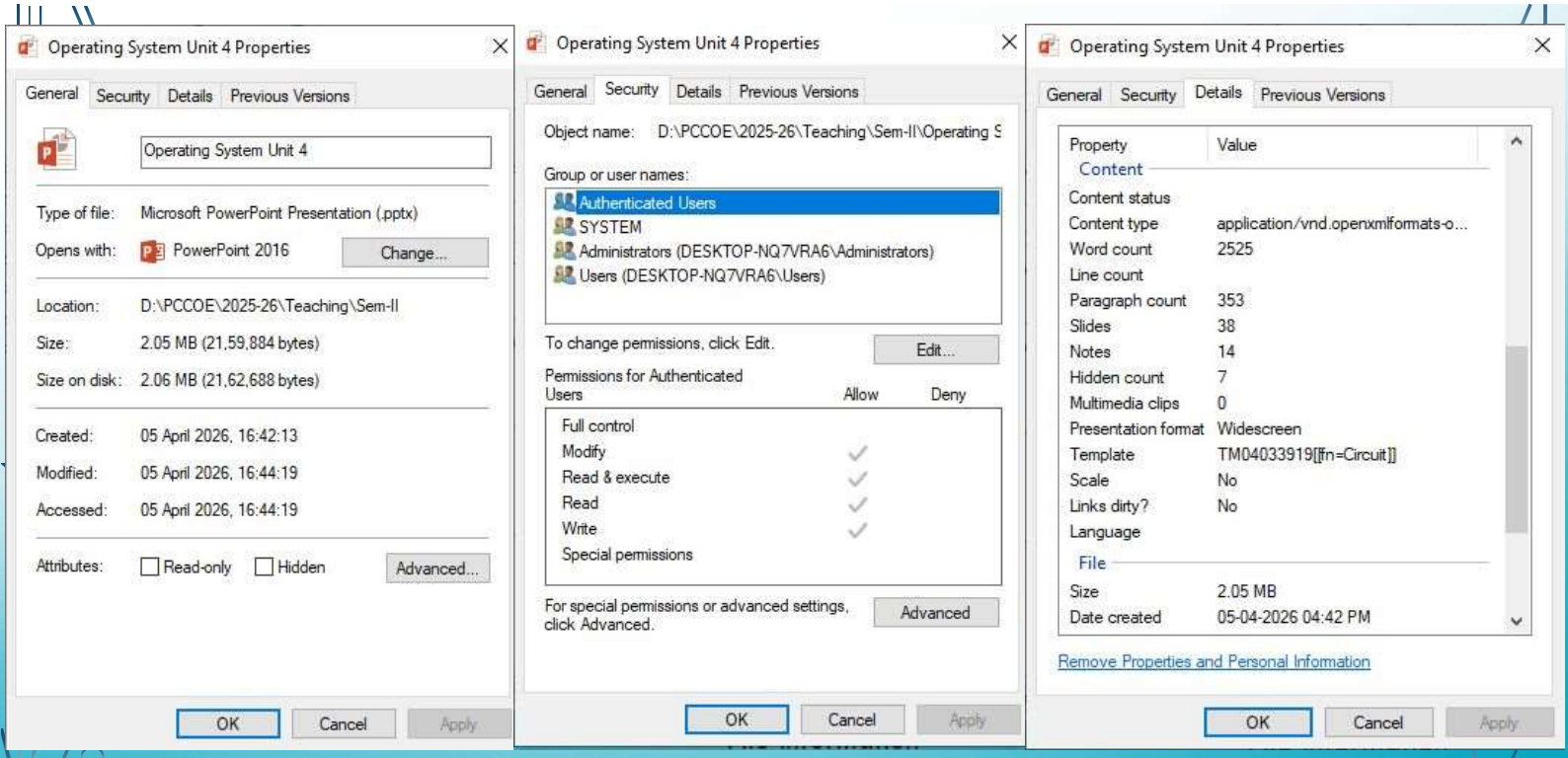
Summative Assessment

- Marks : 30
- Time : 60 minutes

Syllabus units	Marks
Unit 1	7
Unit 2	8
Unit 3	8
Unit 4	7

FILE CONCEPT

- Named collection of data that is manipulated as a unit
- Resides on non-volatile storage
- For user, a file is the smallest allotment of logical secondary storage
- Files represent programs and data
- Creator defines the contents of a file
- A file has a certain defined structure depending on its type
- File Attributes
  - Name, Identifier, Type, Location, Size, Protection, Timestamp, user identification
- File operations
  - Open, Create, Read, Write, Reposition (seek), Delete, Truncate, Rename, Append



File information

File Types

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, perl, asm	source code in various languages
batch	bat, sh	commands to the command interpreter
markup	xml, html, tex	textual data, documents
word processor	xml, rtf, docx	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	gif, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	rar, zip, tar	related files grouped into one file, sometimes compressed, for archiving or storage
multimedia	mpeg, mov, mp3, mp4, avi	binary file containing audio or A/V information

FILE SYSTEMS

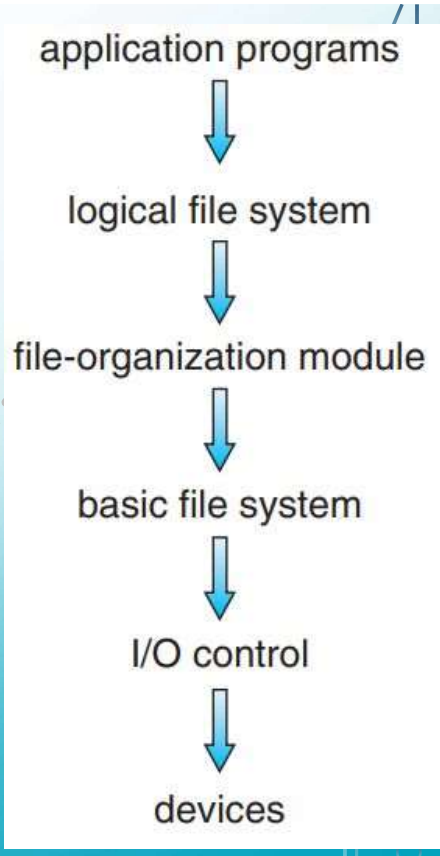
- Organize files and manages access to data
- Responsible for file management, auxiliary storage management, file integrity mechanisms and access methods
- Primarily are concerned with managing secondary storage space, particularly disk storage
- Characteristics:
  - Should exhibit device independence
  - Backup and recovery
  - Encryption and decryption
- Provide efficient and convenient access to the storage device by allowing data to be stored, located, and retrieved easily

- Directory : Files containing the names and locations of other files in the file system, to organize and quickly locate files
- Directory information
- Operation on/in directory
- Single level (or flat) file system
  - All files stored in single directory
- Hierarchical File system
  - Root directory points to other directories
  - Unique files names in a given directory
- Relative path : pathname that does not begin at the root directory
- Absolute path : path beginning at the root
- File structure & Internal File structure

FILE-SYSTEM STRUCTURE

- File systems are maintained on disks
  - A disk can be rewritten in place
  - A disk can access directly any block of information it contains
- File structure
  - Logical storage unit
  - Collection of related information
- I/O transfers in unit of blocks (block is divided into sectors of 512B or 4kB)
- File system design problems:
  - How the file system should look to the user?
  - algorithms and data structures to map the logical file system onto the physical?
- File system has different levels

- **I/O control level** consists of device drivers and interrupt handlers to transfer information between the main memory and the disk system
- **Basic file system** issues generic commands to the device driver to read and write blocks. Manages memory buffers and caches.
- **File-organization module** knows about files and their logical blocks. Includes free-space manager
- **Logical file system** manages metadata information.
  - Maintains file structure via file control blocks (**FCB**)
  - Manages directory structure
  - Also provides protection



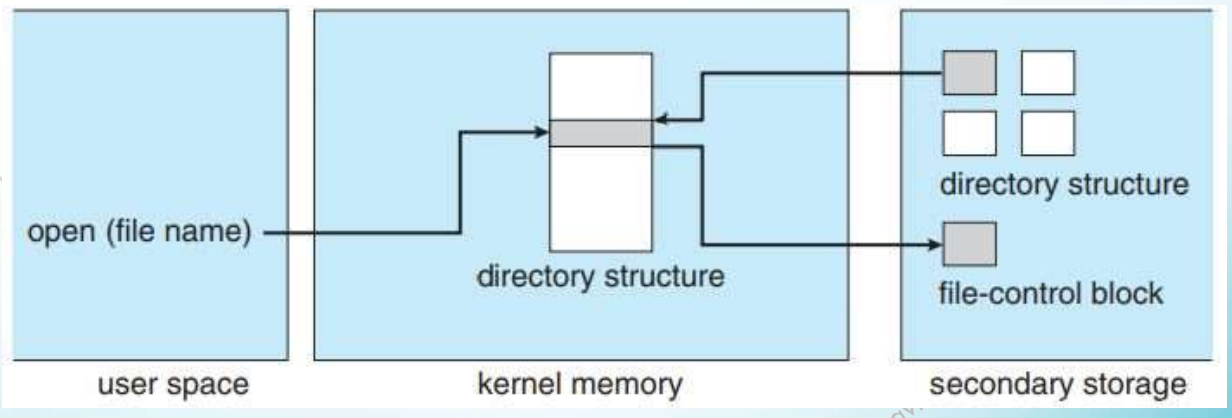
- Windows: FAT, NTFS Unix : UFS
- Linux : ext3, ext4 Google FS, FUSE FS

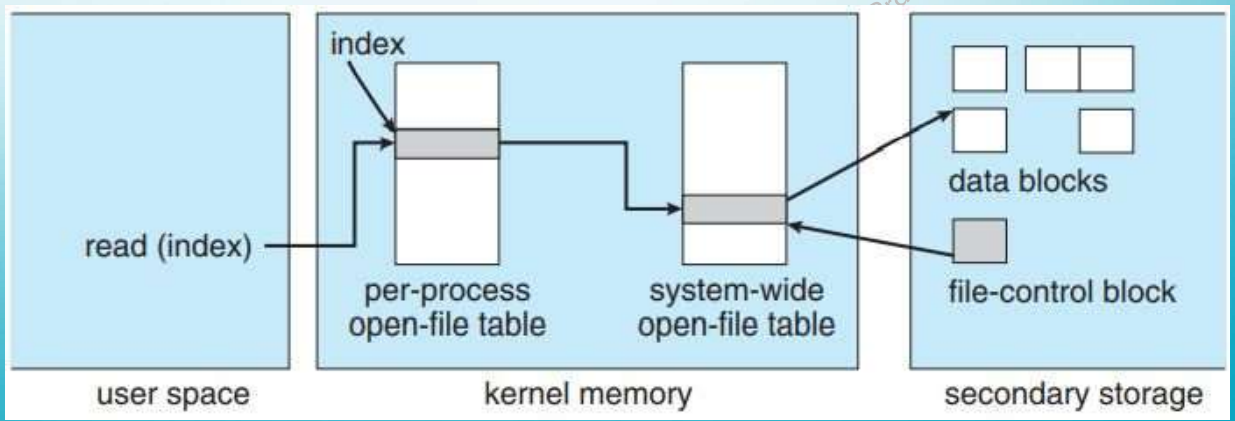
FILE-SYSTEM OPERATIONS

- File system is implemented using on-storage and in-memory structures
- On-storage structures
  - Boot control block (per volume)
  - Volume control block (per volume)
  - Directory structure (per file system)
  - File Control block (per file)
- In-memory
  - Mount table
  - Directory structure caches
  - System-wide open-file table
  - Per-process open-file table
  - Buffers to hold blocks

- A new file creation:
  - A process calls the logical file system (LFS)
  - The LFS knows the format of the directory structures
  - LFS allocates a new FCB
  - System reads the appropriate directory into memory
  - Updates it with the new file name and FCB, and writes it back to the file system
- Using a file:
  - Open()
  - Read()
  - Write()
  - Close()

file permissions
file dates (create, access, write)
file owner, group, ACL
file size
file data blocks or pointers to file data blocks



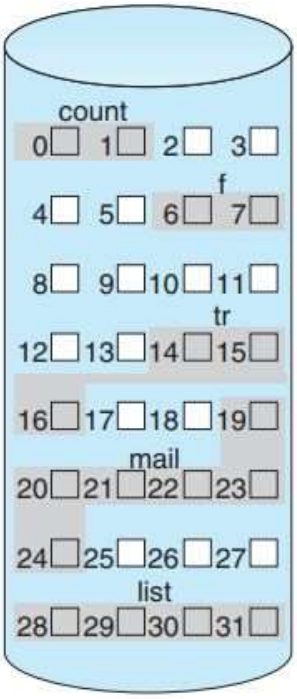


ALLOCATION METHODS

- Direct-access nature of secondary storage provides flexibility in the implementation of files
- How to allocate space with efficient storage utilization and fast access?
  - Contiguous
  - Linked
  - Indexed
- Generally: One method for all files within a file-system type.

CONTIGUOUS ALLOCATION

- File occupies a set of contiguous blocks
- Device addresses define a linear ordering on the device
  - Accessing block b + 1 after block b normally requires no head movement.
  - When needed, head need only move from one track to the next
  - Minimizes seek time
- Defined by the address of the first block and length of the file
- Access is easy:
  - Sequential: remember last referenced and read next one
  - Direct : to read i^{th} block from b, read b+i
- Problems:
  - Finding and determining space for new file
  - External fragmentation



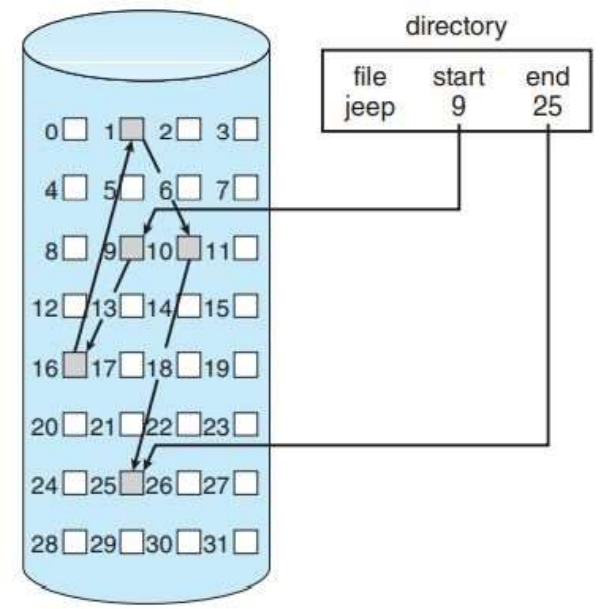
directory

file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

LINKED ALLOCATION

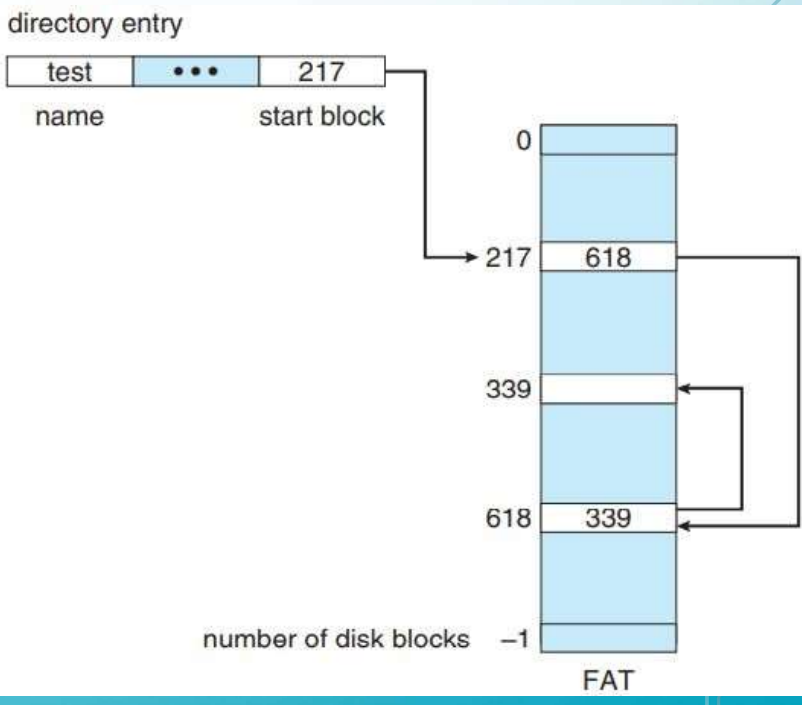
- Solves all problems of contiguous allocation
- A file is a linked list of storage blocks scattered anywhere on the device
- The directory contains a pointer to the first and last blocks of the file.
- Each block contains a pointer to the next block
- **New file creation**:
  - Create a new entry in the directory with pointer to first block
  - Pointer is initialized to null (the end-of-list pointer value) to signify an empty file.
  - The size field is set to 0
  - A write to the file causes system to find a free block, and this new block is written to and is linked to the end of the file.
  - To read, just read blocks by following the pointers from block to block

- Advantages
  - No external fragmentation
  - No need to declare file size at the time of creation
  - File can continue to grow
  - No compaction required
- Disadvantages:
  - Effective only for sequential-access files
  - Pointers take space. File requires little more than actual
  - Solution : clustering of blocks
    - Internal fragmentation
    - For small data, large transfer
  - Reliability: loss/damage of pointer



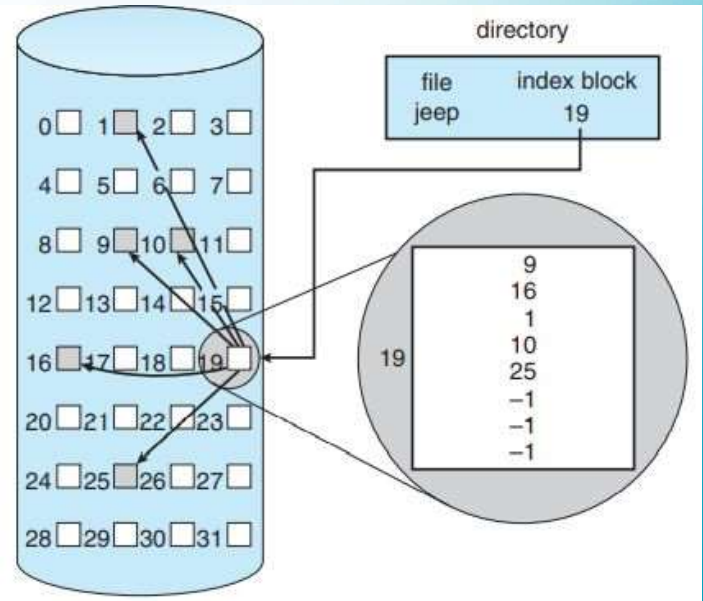
FAT file system

- A variation of linked allocation

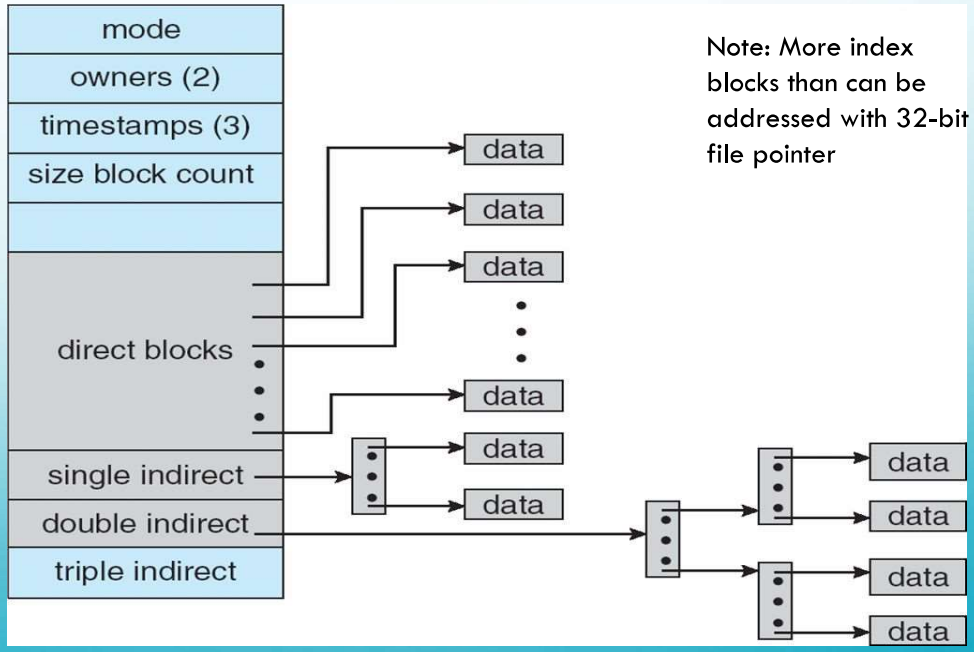


INDEXED ALLOCATION

- Each file has its own index block: an array of storage-block addresses



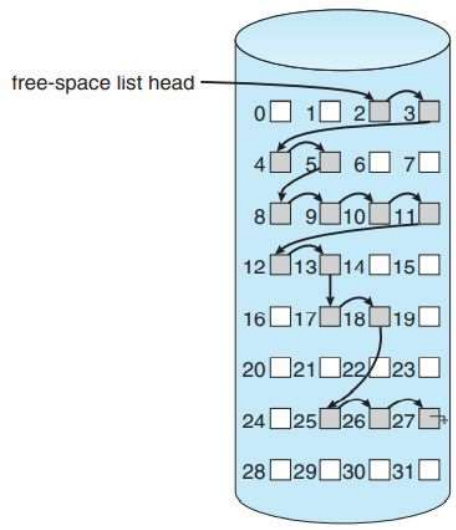
COMBINED SCHEME: UNIX UFS (4K BYTES PER BLOCK, 32-BIT ADDRESSES)



Note: More index blocks than can be addressed with 32-bit file pointer

FREE SPACE MANAGEMENT

- Need to reuse space of deleted files
- OS maintains a free-space list of free blocks
- Bit-vector approach:
  - Aka bitmap
  - Takes 1 bit per block: Free -1, Allocated – 0
  - Ex. 00111100111111000110000000111000
  - Simple & efficient to find first free block or n-consecutive free blocks
- Linked List:
  - Linked list of free block
  - Save pointer to first free block at special location in FS awa cache it in memory
  - Not efficient to traverse the list



- Grouping:
  - stores the addresses of n free blocks in the first free block.
  - The first n−1 of these blocks are actually free.
  - The last block contains the addresses of another n free blocks, and so on
- Counting
  - Store address of first free block and number of free contiguous blocks
- Space maps
  - Used in systems like Oracle ZFS
  - Uses combination of standard enterprise-grade hardware and a unique, storage-optimized operating system (OS) based on the Oracle Solaris kernel with Oracle's ZFS file system at its core.
  - The space map is a log of all block activity (allocating and freeing), in time order, in counting format.

- TRIMing Unused Blocks
  - HDDs: Free block keeps data until overwritten
  - NVM flash-based storage: Do not allow overwrite. Need to erase and then write
  - Mechanism:
    - FS informs storage device that a page is free and can be earased
  - In ATA-attached devices it is called TRIM
  - In NVM-based storage it is *unallocated* command

DISC SCHEDULING ALGORITHMS

- **FIFO/FCFS**
- **SCAN**
- **C-SCAN**
- **SSTF**

Request Queue:

98, 183, 37, 122, 14, 124, 65, 67

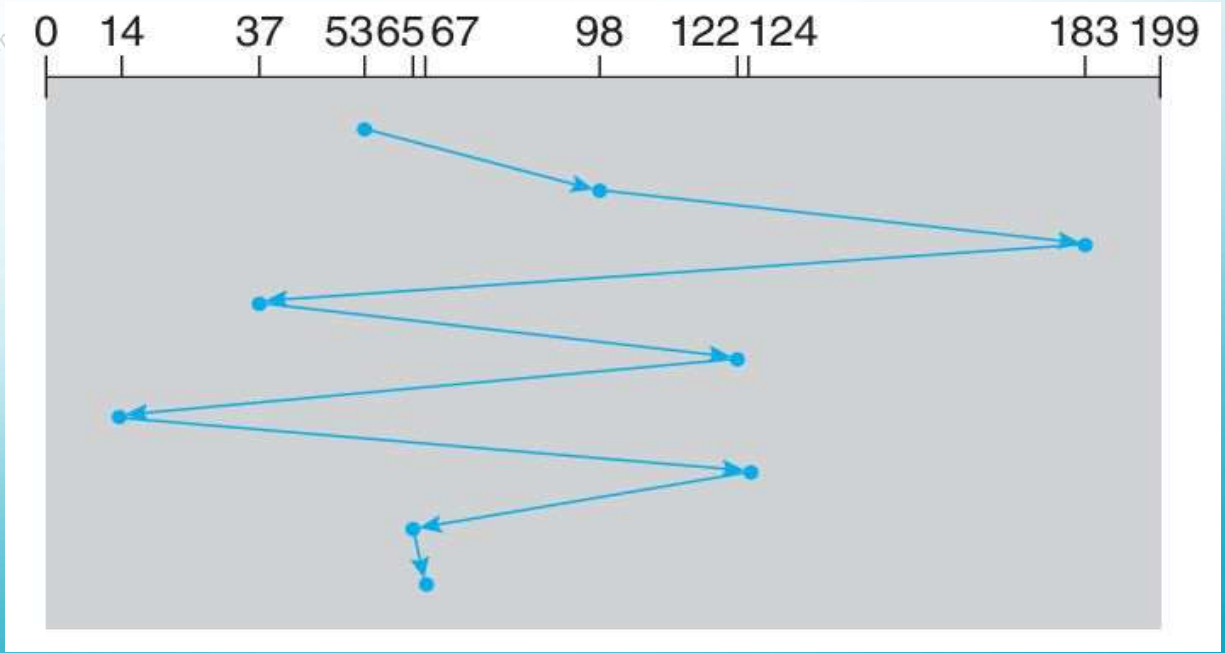
Head position (at cylinder):

53

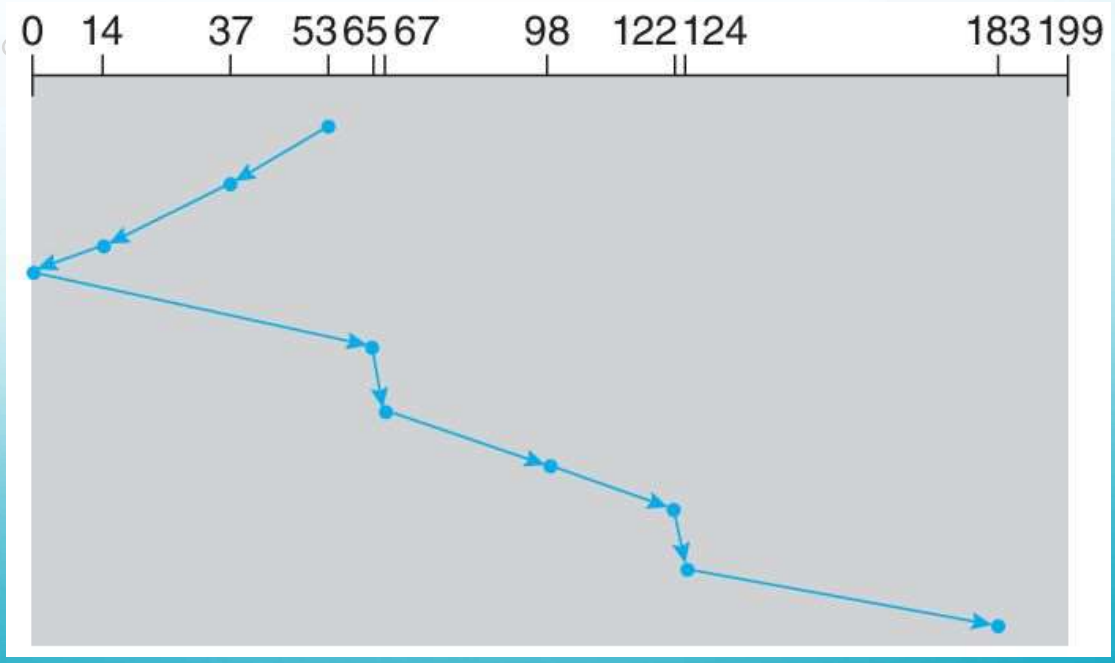
Disk size:

200

First-Come-First-Served

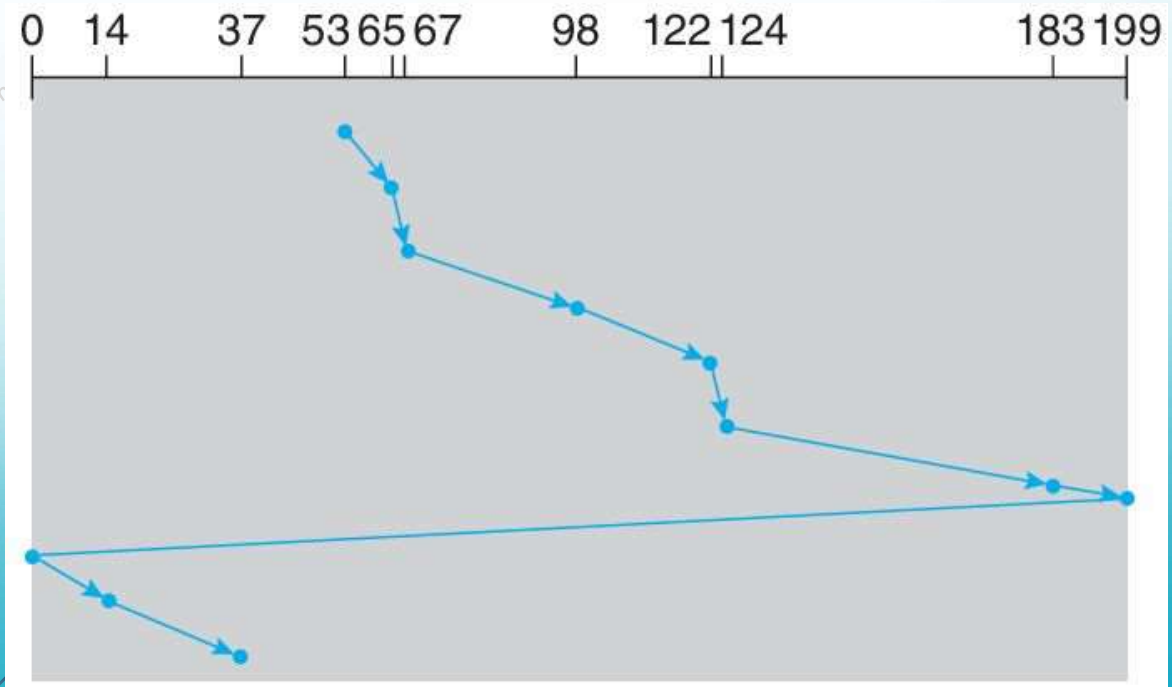


SCAN



Towards 0

C-SCAN



Towards 199

Shortest-Seek-Time-First

53 → 65 → 67
↓
37 → 14
↓
98 → 122 → 124 → 183

Other algorithms

- LOOK
- C-LOOK
- FSCAN
- N-step SCAN
- Deadline
- Anticipatory I/O
- NOOP
- Completely Fair Queuing (CFQ)
- SPTF, SATF

Ex. 1

Consider a disk drive with 4,000 cylinders, numbered from 0 to 3,999. The request queue has the following composition: 1045 750 932 878 1365 1787 1245 664 1678 1897.

If the current position is 1167 and the previous request was served at 1250, compute the total distance (in cylinders) that the disk arm would move for each of the following algorithms: FIFO, SSTF, SCAN, and C-SCAN scheduling.

Ex. 2

Suppose that a disk drive has 5,000 cylinders, numbered 0 to 4,999. The drive is currently serving a request at cylinder 2,150, and the previous request was at cylinder 1,805.

The queue of pending requests, in FIFO order, is:

2,069; 1,212; 2,296; 2,800; 544; 1,618; 356; 1,523; 4,965; 3,681

Starting from the current head position, what is the total distance (in cylinders) that the disk arm moves to satisfy all the pending requests for each of the following disk-scheduling algorithms?

a. FCFS

b. SCAN

c. C-SCAN

d. SSTF

More examples

Given Request queue, initial head, disk size and direction.

Compare performance of FCFS, SSTF, SCAN and C-SCAN

Request queue: 176, 79, 34, 60, 92, 11, 41, 114, 7, 150, 130, 20

Initial head: 50

Disk size: 0–199

Direction: towards 199

Request queue: 10, 180, 20, 170, 30, 160, 40, 150, 50, 140, 60, 130

Initial head: 75

Disk size: 0–199

Direction: towards 0

Request queue: 95, 96, 97, 98, 99, 100, 102, 104, 10, 15, 180, 185, 190

Initial head: 101

Disk size: 0–199

Direction: towards 199

Request queue: 5, 8, 12, 15, 18, 22, 25, 30, 35, 150, 160, 170, 180

Initial head: 90

Disk size: 0–199

Direction: towards 0

Request queue: 20, 40, 60, 80, 100, 120, 140, 160, 180, 10, 30, 50, 70, 90

Initial head: 85

Disk size: 0–199

Direction: towards 199

Request queue: 50, 52, 54, 56, 58, 60, 62, 64, 150, 170, 190, 10, 15

Initial head: 55

Disk size: 0–199

Direction: towards 199

Request queue: 38, 180, 130, 10, 50, 15, 190, 90, 150, 75, 65, 25, 5, 145

Initial head: 70

Disk size: 0–199

Direction: towards 199

Request queue: 12, 85, 190, 45, 60, 5, 155, 130, 75, 25, 95, 110, 180, 20, 140, 65

Initial head: 70

Disk size: 0–199

Direction: towards 199

Request queue: 3, 8, 15, 23, 42, 67, 89, 120, 145, 160, 175, 188

Initial head: 90

Disk size: 0–199

Direction: towards 0

Request queue: 40, 40, 41, 42, 100, 102, 103, 150, 150, 151, 10, 12, 14, 180

Initial head: 50

Disk size: 0–199

Direction: towards 199

Request queue: 0, 1, 2, 3, 4, 195, 196, 197, 198, 199, 120, 130, 140

Initial head: 100

Disk size: 0–199

Direction: towards 199

Request queue: 182, 45, 78, 10, 90, 150, 30, 60, 15, 200, 175, 5, 95, 140, 120, 35

Initial head: 85

Disk size: 0–200

Direction: towards 199

REFERENCES

- Silberschatz, Galvin, Gagne, "Operating System Principles", 10th Edition
- Deitel, Deitel, Choffnes, “Operating System”, 3rd edition, Pearson